

# RESTfull-API

with pure Django

7/26/2025

Maxim Danilov, danilovmy



# Who I am



- Professional skier 🏂
- Trail runner 🏃
- Sommelier 🍷🍷
- Mentor for developer community for free 🧐.  
(and not free for companies).

# Thanks for support for my family





# Thanks Maja & Mark for illustrations



7/26/2025

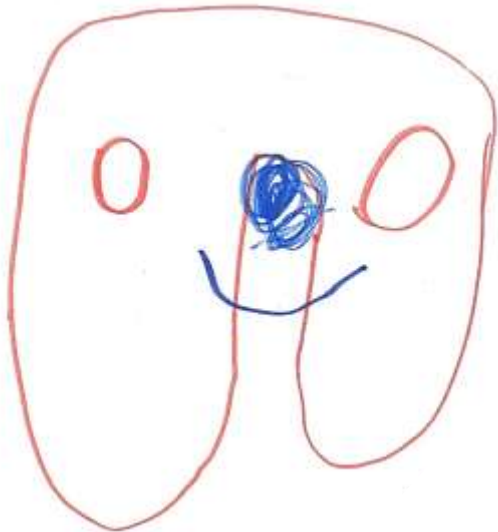


Maxim Danilov, danilovmy

# Links

<https://github.com/danilovmy/pyconpt2025/tree/main/workshop>

Slides & code samples from this workshop



# What is REST (Re-presentative State Transfer)

- 6 Principles, how server application can be built.
- Author Roy Fielding, 2000



# REST principles (1 from 6)

- Server independent from clients



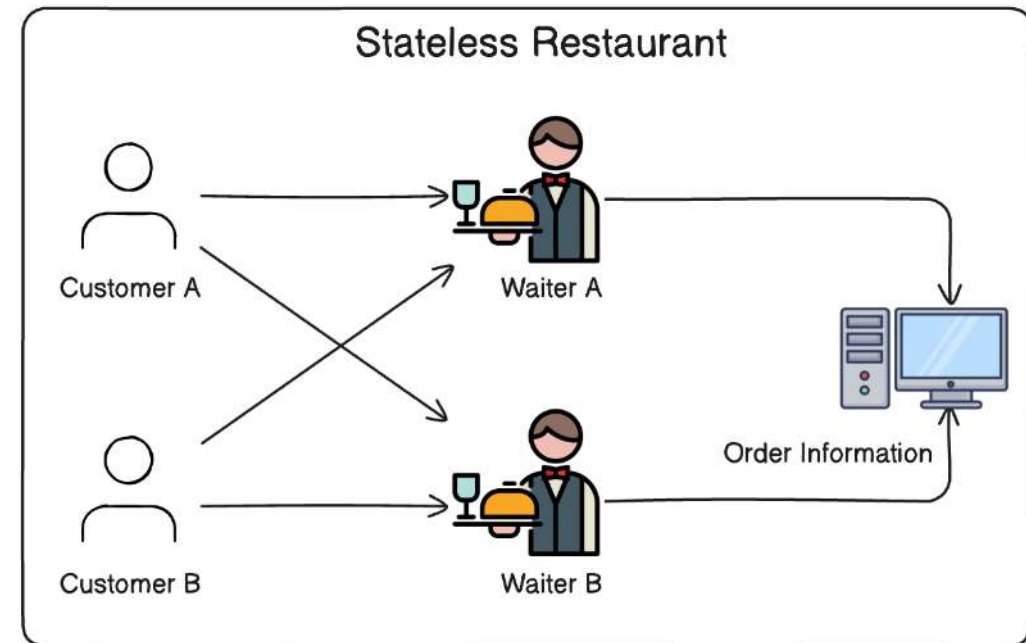
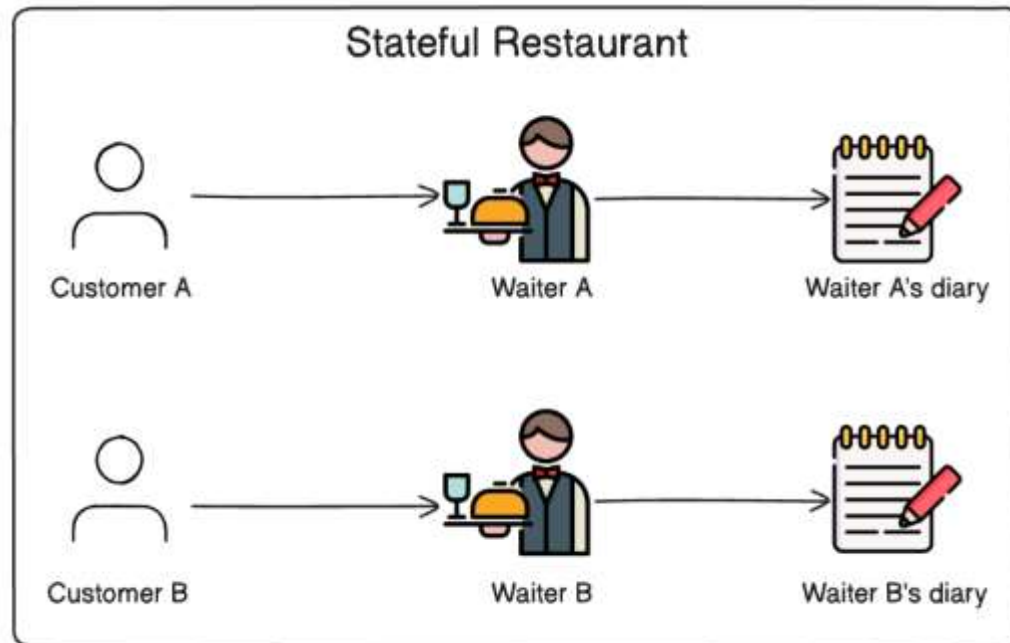
7/26/2025





# REST principles (2 from 6)

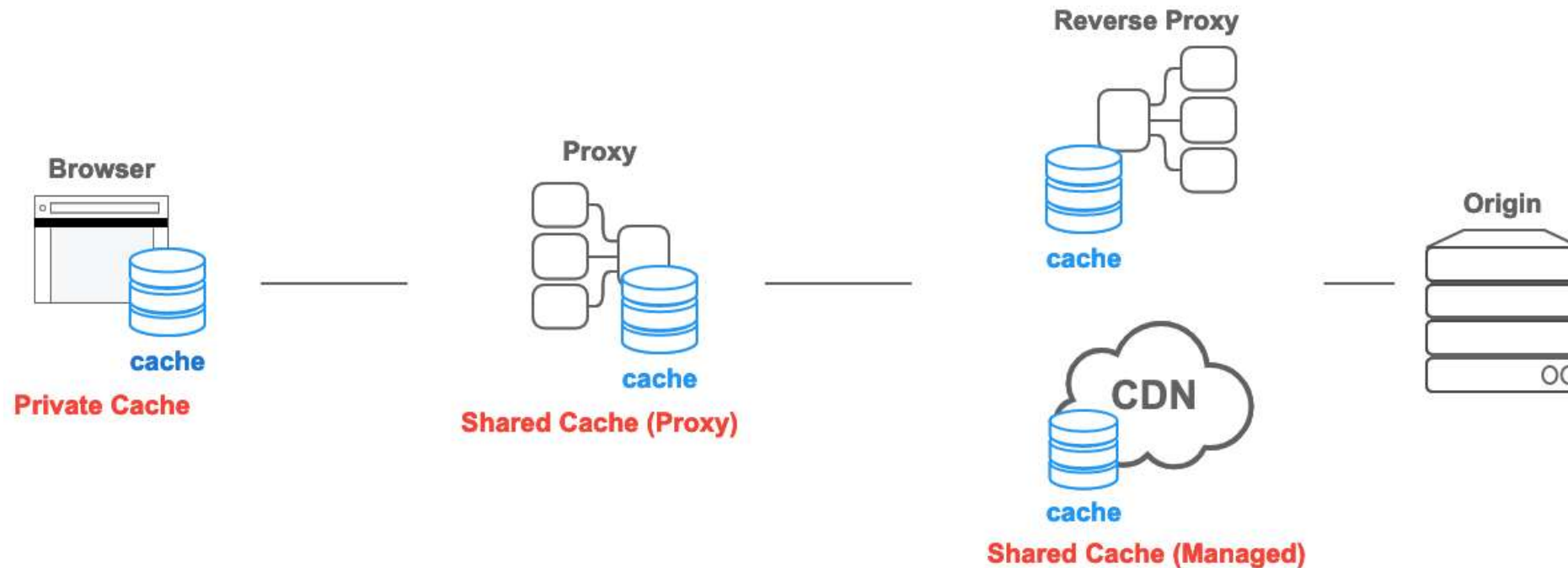
- Statelessness.





# REST principles (3 from 6)

- Cacheability.



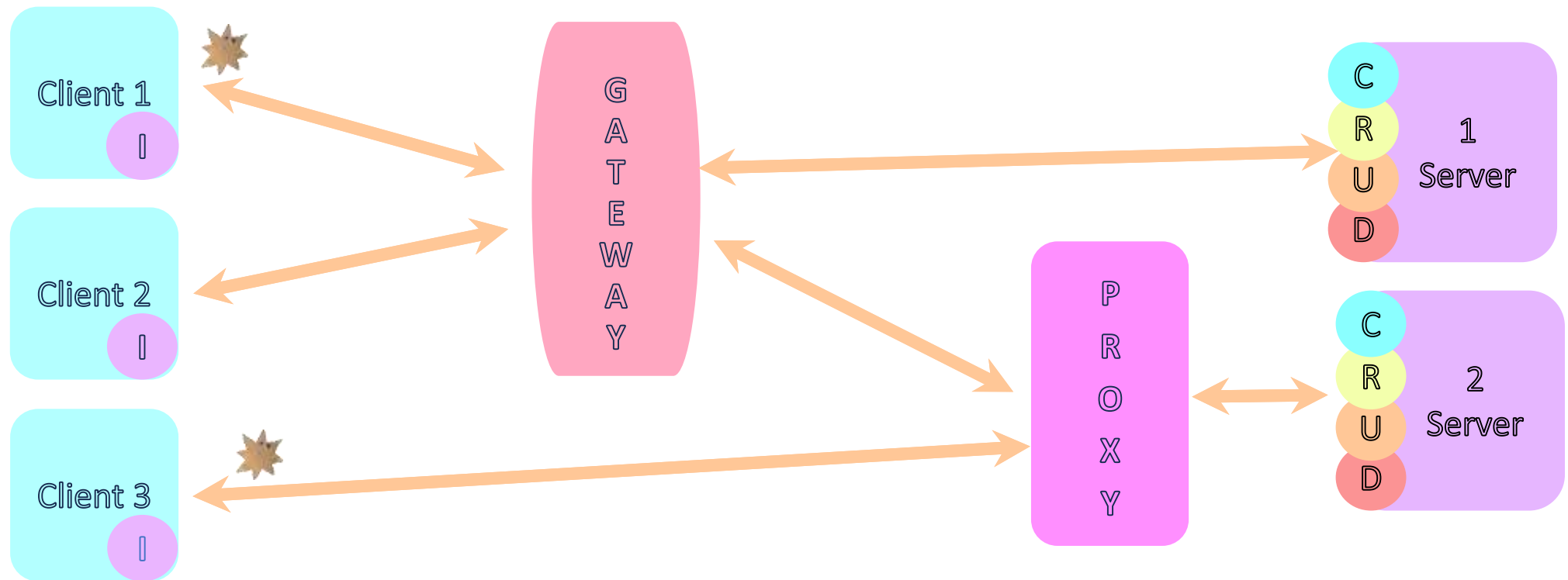
# REST principles (4 from 6)

- Uniform interface.



# REST principles (5 from 6)

- Layered server system.



# REST principles (6 from 6)

- **Optional** code-on-demand

7.1, RFC 4329. **obsolete**

GET /product/id HTTP/1.1

Content-Type: content-type: text/javascript; charset=utf-8

7.2, RFC 4329

GET /product/id HTTP/1.1

Content-Type: content-type: application/javascript; charset=utf-8





# REST principles (>6 from 6)

- Any other suggestions are just an extension of Roy Fielding's original idea

A good extension for REST documentation from AWS  
<https://aws.amazon.com/what-is/restful-api/>



# RESTfull api

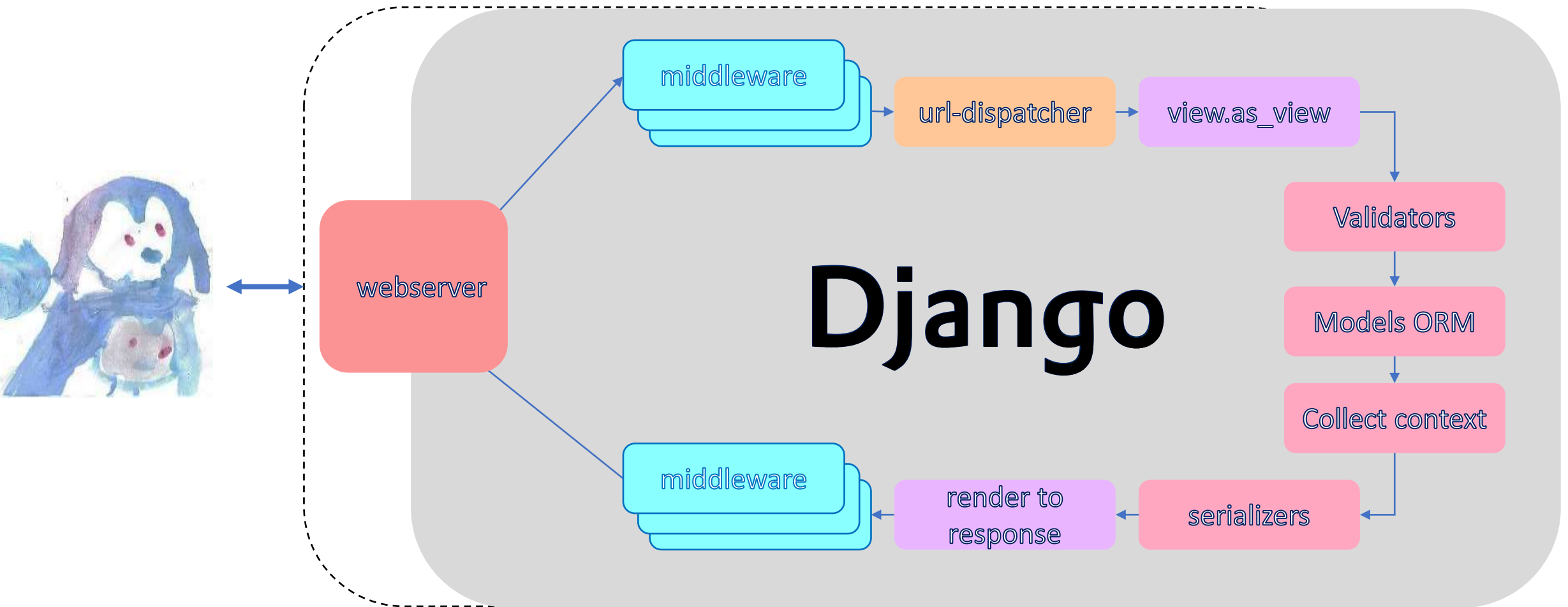
- APIs following the REST architectural style are called **REST APIs**
- Web services containing REST APIs are **RESTful web services**.
- Under **RESTful APIs** are meant **RESTful web services**.



# APIs in Django are not RESTfull...

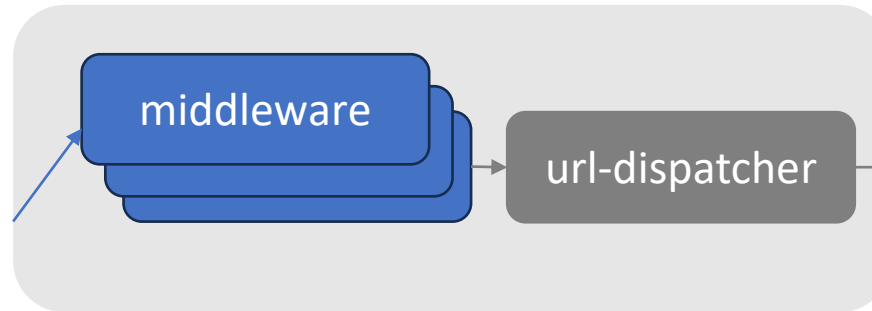
- Generally, stateful due sessions
- The Routing for Request Methods not in a box
- Handlers for TRACE, HEADS, PATCH request methods not in the box
- API autodocumentation is not in the box

# APIs elements in Django:



# APIs elements in Django:

Middleware in Django:

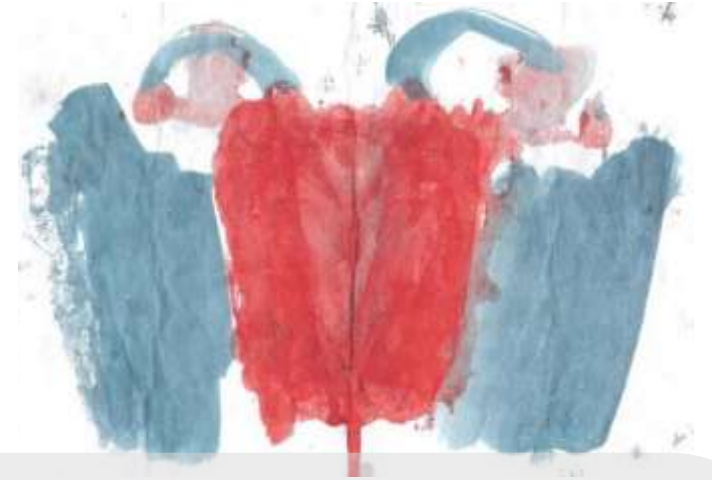


1. Applied on every request
2. Reduced performance
3. Breaks statelessy
4. Not every middleware suited for every request





# Middleware alternatives:



- method / function decorator

```
from django.utils.decorators import method_decorator
from django.utils.decorators import decorator_from_middleware_with_args

cache_func = decorator_from_middleware_with_args(CacheMiddleware)

@method_decorator('get', cache_func(3600))
def ProductDetail(DetailView):
    ...
```

# What's new in Django 5.2

## Decorators 🌱

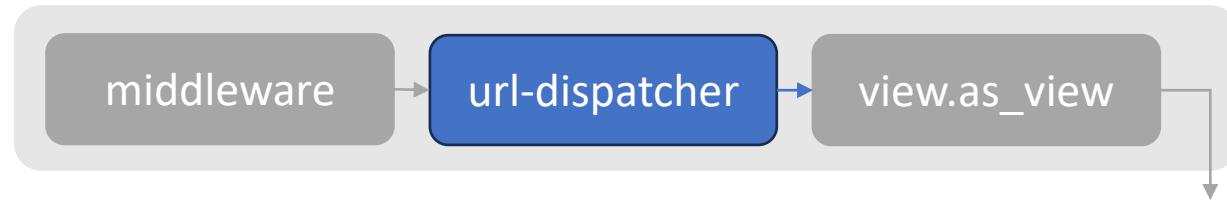
- `method_decorator()` now supports wrapping asynchronous view methods.

```
@method_decorator('get', cache_func(3600))
def ProductDetail(DetailView):
    ...
    async def get(self, *args, **kwargs):
        ...
```

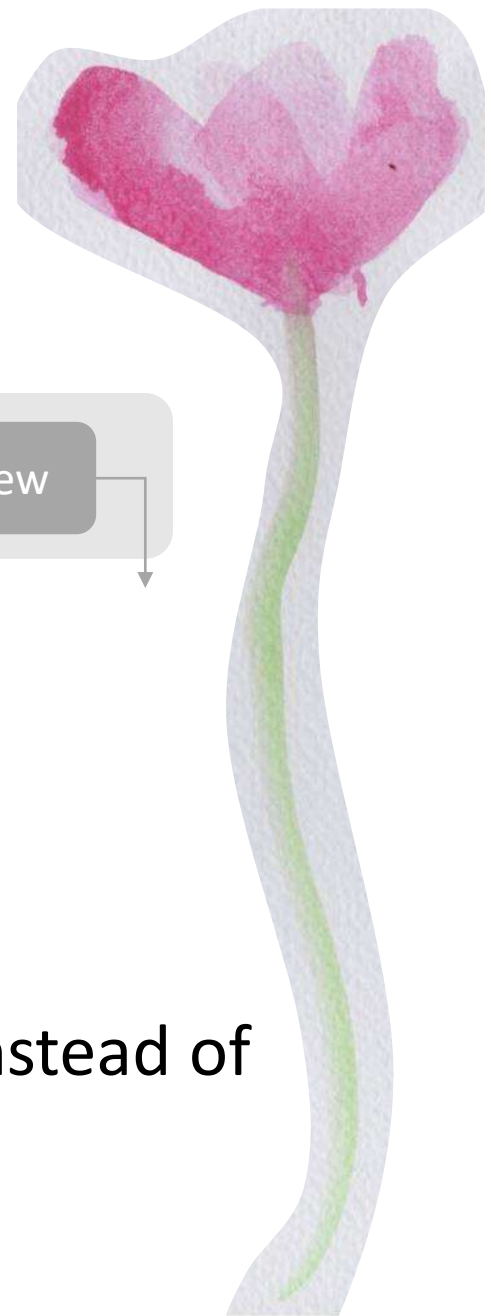


# APIs elements in Django:

Url dispatcher in Django:



1. Class **URLResolver** aka “urlpatterns”
2. Provide appropriated handler by URI from request
3. Single url or group of urls can be wrapped by decorator instead of using middleware.



# Middleware alternatives:

wrappers for groups of urls (still not async)

```
from authlib.integrations.django_oauth2 import ResourceProtector

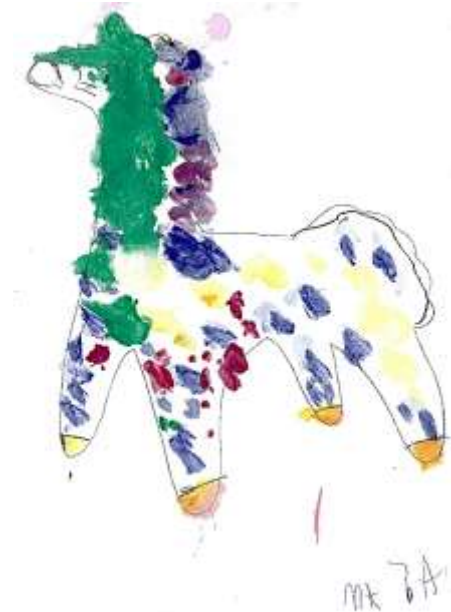
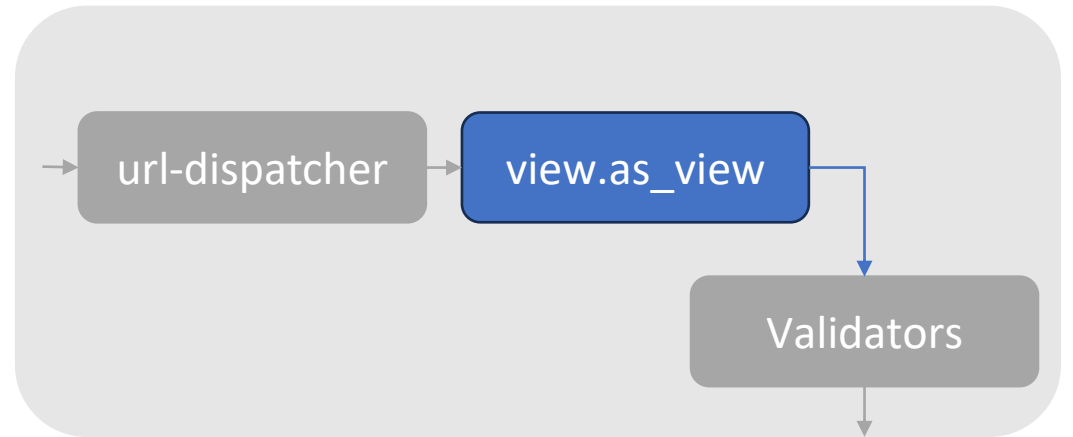
jwt_protector = ResourceProtector()

urlpatterns = [path('api/', jwt_protector(include('api.urls')))]
```



# APIs elements in Django:

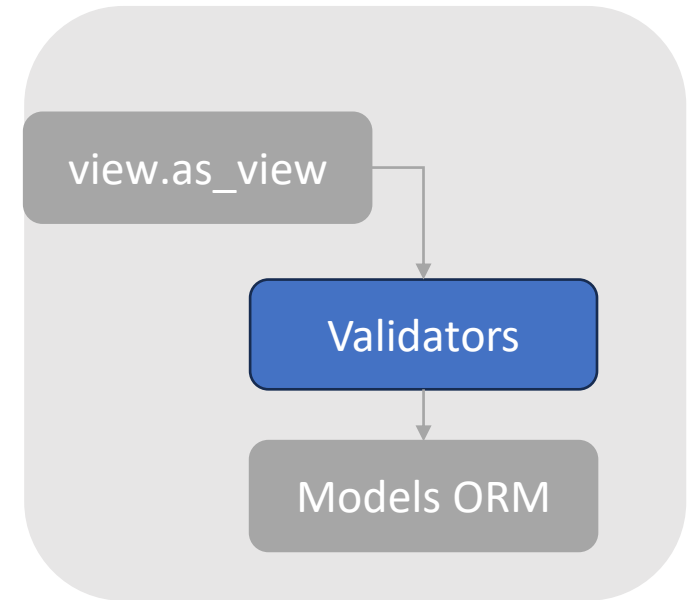
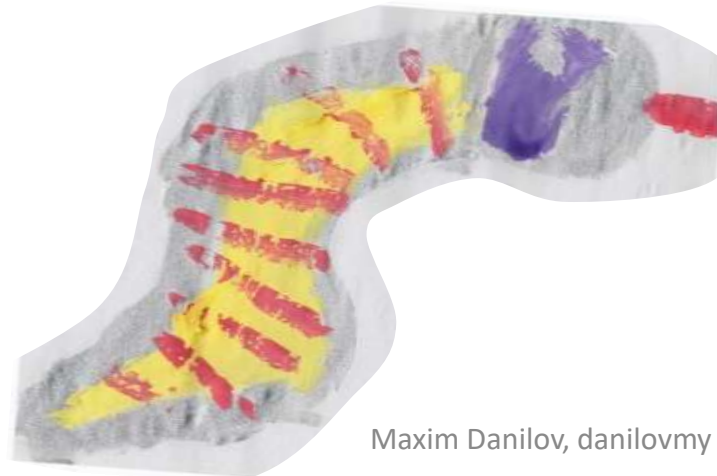
- sync/async FB views or GCB views
- `GCBView.dispatch` Provide handler for appropriate request.method
- response object should be returned





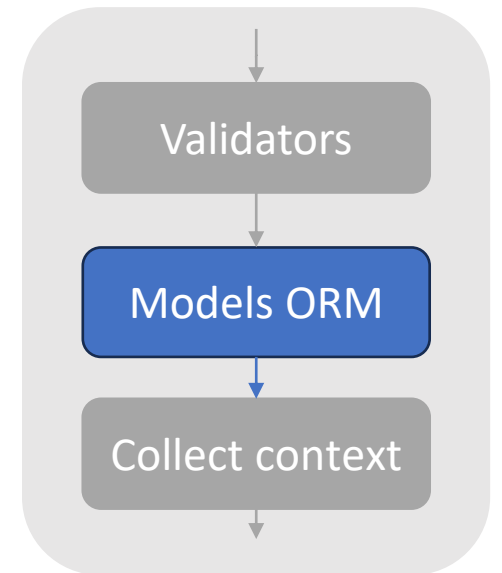
# APIs elements in Django:

- Django.forms.Form as a data validator.
- Simple data validation with Validators
- Complex validation on Form.\_post\_clean()



# APIs elements in Django:

- `Model.refresh_from_db()` – to fetch data from DB
- `Model.save()` – to save data to DB
- Complex DB validation on `Model.full_clean()`
  - `Model.clean()`
  - `Model.validate_unique()`
- Some methods formally a-sync



# What's new in Django 5

## Signals 🐦

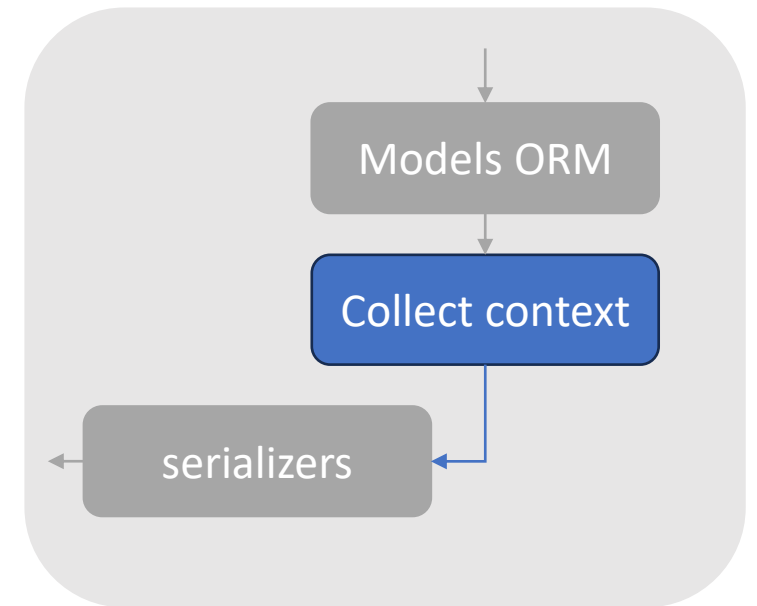
- The new `Signal.send()` and `Signal.send_robust()` methods allow asynchronous signal dispatch. Signal receivers may be synchronous or asynchronous, and will be automatically adapted to the correct calling style.

```
@receiver(my_signal)
async def my_async_receiver(sender, **kwargs):
    await logging.info(f"{sender} signal received")
```

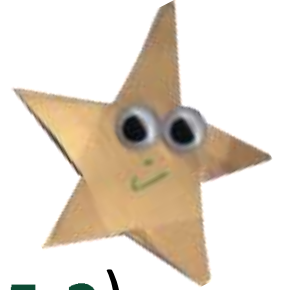


# APIs elements in Django:

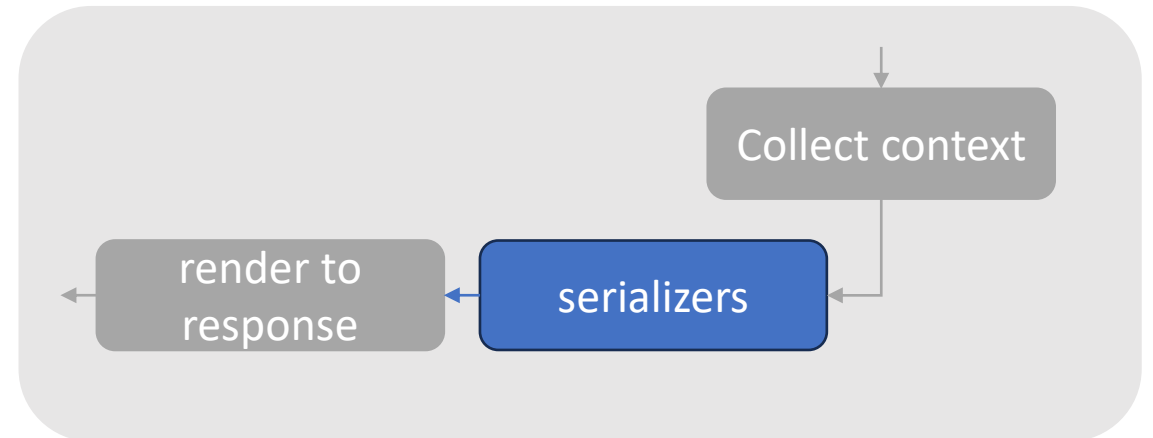
- Weakest part in GCBV, not designed to be async
- Designed to perform final business logic



# APIs elements in Django:

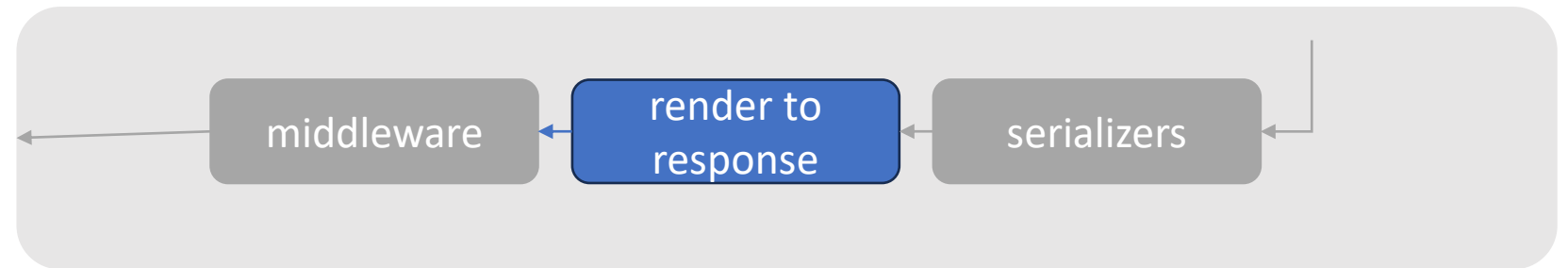


- Django.core.serializer as a result serializer, (improvements in **Dj5.2**)
- Knows, how to serialize specific data.
- Should not perform data validation
- Not designed to be async!



# APIs elements in Django:

- JsonResponse / HttpResponse class for response.
- StreamingHttpResponse for partially prepared data (async)

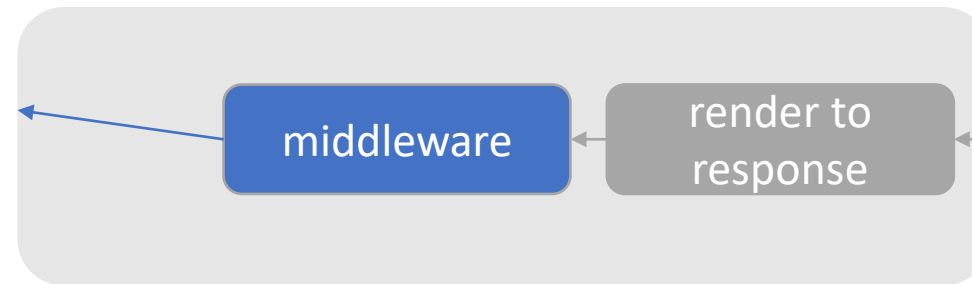




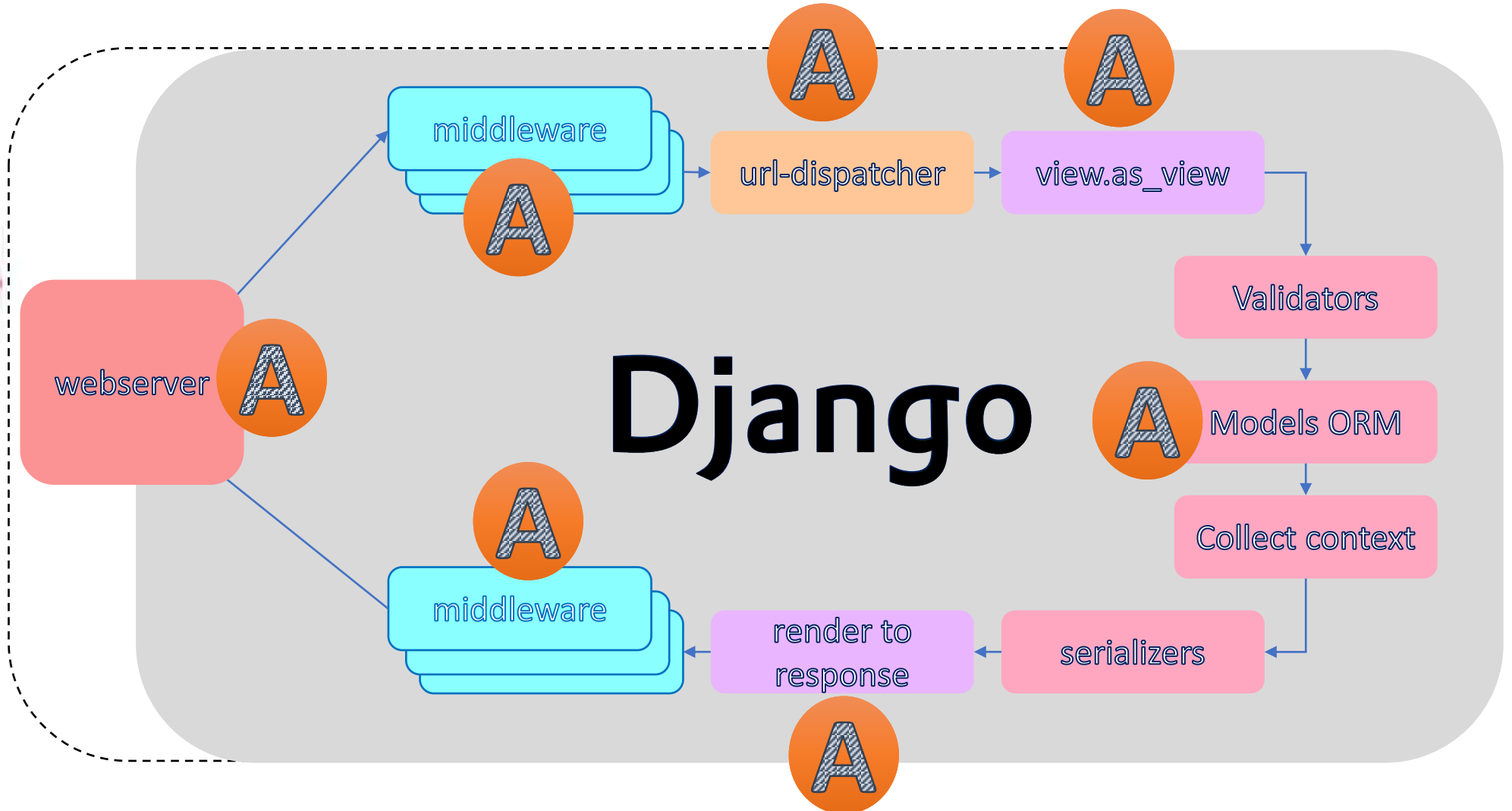
# APIs elements in Django:

Last middleware in Django „convert\_exception\_to\_response”.

To serialize Exception from Django.core.exceptions in Response.



# Async APIs elements in Django:

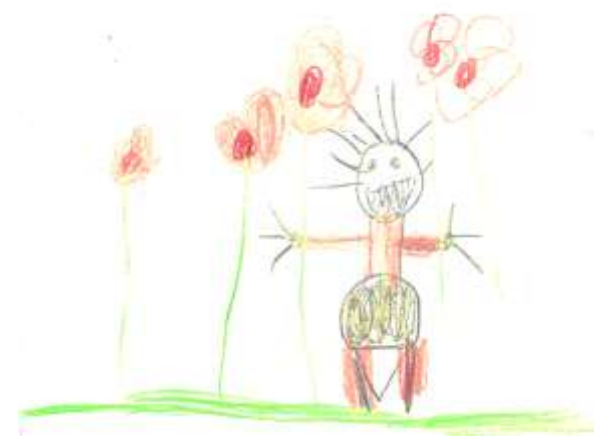


# What's new in Django 5

- Auth/permissions decorator (in general async)
  - The new asynchronous functions are now provided, using an **a** prefix: `django.contrib.auth.aauthenticate()`, `aget_user()`, `alogin()`, `alogout()`, and `update_session_auth_hash()`.
  - **AuthenticationMiddleware** now adds an `HttpRequest.auser()` asynchronous method that returns the currently logged-in user.
- Cache decorators (async in last version)

## Decorators

- The following decorators now support wrapping asynchronous view functions:
  - `cache_control()`



# REST API Autodocumentation:

- Request.method.**OPTION** to provide yaml documentation about the API
- Handler.**doctring** used to provide information for yaml generation
- dockstring is updated automatically by the **linter**



[www.youtube.com/watch?v=MeoKMdyPRWo](https://www.youtube.com/watch?v=MeoKMdyPRWo)



# Summary:

- Raw Django has EVERYTHING you need to create REST APIs
- REST APIs in Raw Django can be a synchronous or asynchronous
- Raw Django is not ready to offer full async functionality
- Know your framework for fun and profit!

# Thank you for your attention!

**Maxim Danilov**

**maxim@wpshop.at**



**Link to slides & code**

<https://github.com/danilovmy/pyconpt2025/tree/main/workshop>

